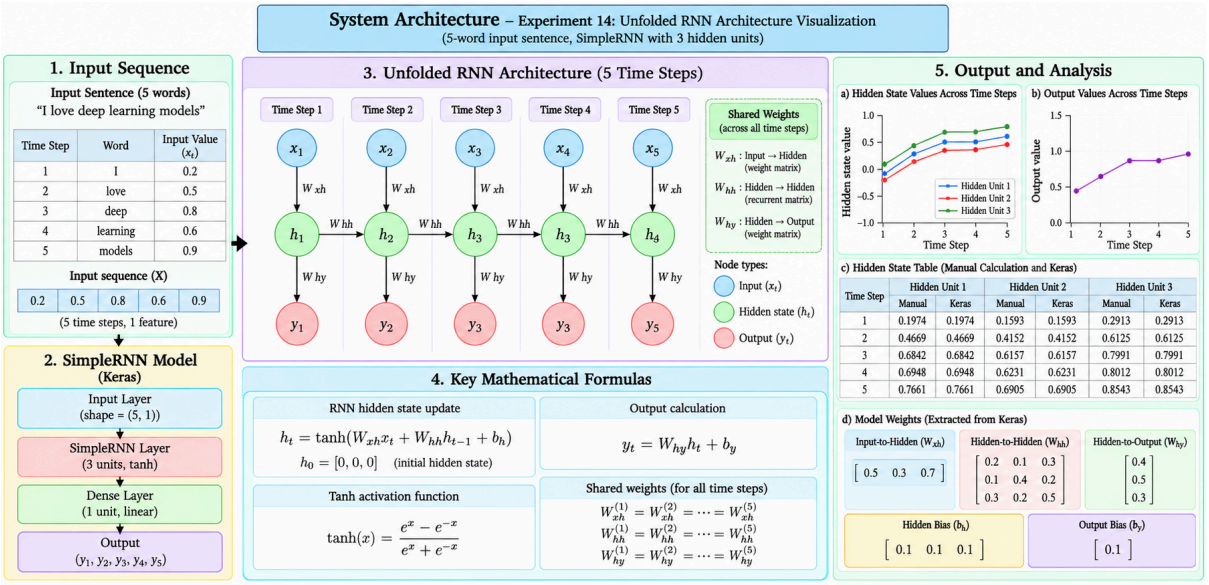
Experiment 14: Unfolded RNN Architecture Visualization

1. Aim

To build and visualize a Simple Recurrent Neural Network (SimpleRNN) using TensorFlow and Keras by unfolding its recurrent structure across five time steps. To understand how input values, hidden states, and shared weights flow through the network, manually calculate hidden-state values at each time step, verify them against Keras outputs, represent the unfolded architecture using NetworkX, and plot hidden-state variations over time using Matplotlib.



2. Mathematical Formulas

1. RNN Hidden State Calculation

$$\boxed{h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)}$$

Where:

- x_t = Input at time step t.
- h_t = Hidden state at time step t.
- W_{xh} = Input-to-hidden weight.
- W_{hh} = Hidden-to-hidden recurrent weight.
- b_h = Hidden layer bias.

2. Initial Hidden State

$$\boxed{h_0 = [0, 0, 0]}$$

3. Tanh Activation Function

$$\boxed{\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}}$$

4. Output Calculation

$$\boxed{y_t = W_{hy}h_t + b_y}$$

Where W_{hy} represents the hidden-to-output weight.

5. Hidden State for Five Time Steps

$$h_1 = \tanh(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

$$h_2 = \tanh(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

$$h_3 = \tanh(W_{xh}x_3 + W_{hh}h_2 + b_h)$$

$$h_4 = \tanh(W_{xh}x_4 + W_{hh}h_3 + b_h)$$

$$h_5 = \tanh(W_{xh}x_5 + W_{hh}h_4 + b_h)$$

6. Shared Weights

$$\boxed{W_{xh}^{(1)} = W_{xh}^{(2)} = \cdots = W_{xh}^{(5)}}$$

```python
# Experiment 14: Unfolded RNN Architecture Visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import networkx as nx

np.set_printoptions(precision=4, suppress=True)

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. INPUT SENTENCE
# ========================================

words = ["I", "love", "deep", "learning", "models"]

# Illustrative numerical representation of words
X = np.array([
    [0.2],
    [0.5],
    [0.8],
    [0.6],
    [0.9]
], dtype=np.float32)

X_input = X.reshape(1, 5, 1)

print("Input Sentence:", " ".join(words))
print("\nInput Values:")
print(X.ravel())

# ========================================
# 2. BUILD SIMPLERNN MODEL
# ========================================

inputs = tf.keras.Input(shape=(5, 1))

rnn_layer = tf.keras.layers.SimpleRNN(
    3,
    activation="tanh",
    return_sequences=True,
    name="simple_rnn"
)

hidden_states = rnn_layer(inputs)

output_layer = tf.keras.layers.Dense(
    1,
    activation="linear",
    name="output"
)

outputs = output_layer(hidden_states)

# Modified to output both hidden states and final outputs to allow unpack during prediction
model = tf.keras.Model(inputs, [hidden_states, outputs])

# Build the model
model(X_input)
```

```
# ========================================
# 3. SET FIXED WEIGHTS
# ========================================

W_xh = np.array([
    [0.5, 0.3, 0.7]
], dtype=np.float32)

W_hh = np.array([
    [0.2, 0.1, 0.3],
    [0.1, 0.4, 0.2],
    [0.3, 0.2, 0.5]
], dtype=np.float32)

b_h = np.array([0.1, 0.1, 0.1], dtype=np.float32)

W_hy = np.array([
    [0.4],
    [0.5],
    [0.3]
], dtype=np.float32)

b_y = np.array([0.1], dtype=np.float32)

# Assign weights to Keras layers
rnn_layer.set_weights([W_xh, W_hh, b_h])
output_layer.set_weights([W_hy, b_y])

print("\nInput-to-Hidden Weights:")
print(W_xh)

print("\nHidden-to-Hidden Weights:")
print(W_hh)

print("\nHidden Bias:")
print(b_h)

print("\nHidden-to-Output Weights:")
print(W_hy)

# ========================================
# 4. MANUAL HIDDEN STATE CALCULATION
# ========================================

h_prev = np.zeros((1, 3))

manual_hidden_states = []
manual_outputs = []

print("\nMANUAL HIDDEN STATE CALCULATIONS")

for t in range(5):

    x_t = X_input[:, t, :]

    # RNN calculation
    z = x_t @ W_xh + h_prev @ W_hh + b_h

    h_t = np.tanh(z)

    # Output calculation
    y_t = h_t @ W_hy + b_y

    manual_hidden_states.append(h_t[0].copy())
```

```
    manual_outputs.append(y_t[0, 0])

    print(f"\nTime Step {t+1}")
    print("Input:", x_t.ravel())
    print("Hidden State:", h_t.ravel())
    print("Output:", y_t.ravel())

    h_prev = h_t

manual_hidden_states = np.array(manual_hidden_states)
manual_outputs = np.array(manual_outputs)

# ========================================
# 5. VERIFY WITH KERAS
# ========================================

keras_hidden, keras_outputs = model.predict(
    X_input, verbose=0
)

keras_hidden = keras_hidden[0]
keras_outputs = keras_outputs[0].ravel()

print("\nKERAS HIDDEN STATES:")
print(keras_hidden)

print("\nMANUAL HIDDEN STATES:")
print(manual_hidden_states)

print("\nKERAS OUTPUTS:")
print(keras_outputs)

print("\nMANUAL OUTPUTS:")
print(manual_outputs)

print(
    "\nMaximum Hidden-State Difference:",
    np.max(np.abs(keras_hidden - manual_hidden_states))
)

print(
    "Maximum Output Difference:",
    np.max(np.abs(keras_outputs - manual_outputs))
)

# ========================================
# 6. DRAW UNFOLDED RNN USING NETWORKX
# ========================================

G = nx.DiGraph()

positions = {}

# Create input, hidden and output nodes
for t in range(5):

    input_node = f"x{t+1}"
    hidden_node = f"h{t+1}"
    output_node = f"y{t+1}"

    G.add_node(input_node)
    G.add_node(hidden_node)
    G.add_node(output_node)
```

```
    positions[input_node] = (t, 2)
    positions[hidden_node] = (t, 1)
    positions[output_node] = (t, 0)

    # Input-to-hidden connection
    G.add_edge(
        input_node, hidden_node,
        label="W_xh"
    )

    # Hidden-to-output connection
    G.add_edge(
        hidden_node, output_node,
        label="W_hy"
    )

    # Recurrent hidden connection
    if t > 0:
        G.add_edge(
            f"h{t}", hidden_node,
            label="W_hh"

        )

plt.figure(figsize=(14, 7))

nx.draw_networkx_nodes(
    G, positions,
    nodelist=[f"x{i}" for i in range(1, 6)],
    node_color="skyblue",
    node_size=1600
)

nx.draw_networkx_nodes(
    G, positions,
    nodelist=[f"h{i}" for i in range(1, 6)],
    node_color="lightgreen",
    node_size=1600
)

nx.draw_networkx_nodes(
    G, positions,
    nodelist=[f"y{i}" for i in range(1, 6)],
    node_color="salmon",
    node_size=1600
)

nx.draw_networkx_edges(
    G, positions,
    arrows=True,
    arrowsize=20,
    node_size=1600,
    connectionstyle="arc3,rad=0.05"
)

nx.draw_networkx_labels(
    G, positions,
    font_size=12,
    font_weight="bold"
)

edge_labels = nx.get_edge_attributes(G, "label")

nx.draw_networkx_edge_labels(
```

```
    G, positions,
    edge_labels=edge_labels,
    font_size=9,
    label_pos=0.5
)

plt.text(2, 2.5, "INPUT LAYER",
         ha="center", fontsize=12, fontweight="bold")

plt.text(2, 1.5, "HIDDEN STATE LAYER",
         ha="center", fontsize=12, fontweight="bold")

plt.text(2, -0.5, "OUTPUT LAYER",
         ha="center", fontsize=12, fontweight="bold")

plt.title(
    "Unfolded SimpleRNN Architecture - 5 Time Steps",
    fontsize=15
)

plt.axis("off")
plt.tight_layout()
plt.show()

# ========================================
# 7. PLOT HIDDEN STATES ACROSS TIME
# ========================================

plt.figure(figsize=(9, 5))

for i in range(3):
    plt.plot(
        range(1, 6),
        manual_hidden_states[:, i],
        marker="o",
        label=f"Hidden Unit {i+1}"
    )

plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.title("Hidden State Values Across Five Time Steps")
plt.xticks(range(1, 6))
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 8. DISPLAY HIDDEN STATE TABLE
# ========================================

hidden_table = pd.DataFrame(
    manual_hidden_states,
    columns=["Hidden Unit 1", "Hidden Unit 2", "Hidden Unit 3"]
)

hidden_table.insert(
    0, "Time Step", range(1, 6)
)

print("\nHIDDEN STATE TABLE:")
print(hidden_table.round(4))

print("\nExperiment Completed Successfully!")
```

OUTPUT

```
Input Sentence: I love deep learning models

Input Values:
[0.2 0.5 0.8 0.6 0.9]

Input-to-Hidden Weights:
[[0.5 0.3 0.7]]

Hidden-to-Hidden Weights:
[[0.2 0.1 0.3]
 [0.1 0.4 0.2]
 [0.3 0.2 0.5]]

Hidden Bias:
[0.1 0.1 0.1]

Hidden-to-Output Weights:
[[0.4]
 [0.5]
 [0.3]]

MANUAL HIDDEN STATE CALCULATIONS

Time Step 1
Input: [0.2]
Hidden State: [0.1974 0.1586 0.2355]
Output: [0.3289]

Time Step 2
Input: [0.5]
Hidden State: [0.443  0.363  0.5775]
Output: [0.6319]

Time Step 3
Input: [0.8]
Hidden State: [0.663  0.5683 0.8192]
Output: [0.8951]

Time Step 4
Input: [0.6]
Hidden State: [0.6832 0.6276 0.8461]
Output: [0.9409]

Time Step 5
Input: [0.9]
Hidden State: [0.7629 0.6955 0.9021]
Output: [1.0236]
WARNING:tensorflow:5 out of the last 58 calls to <function Tenso

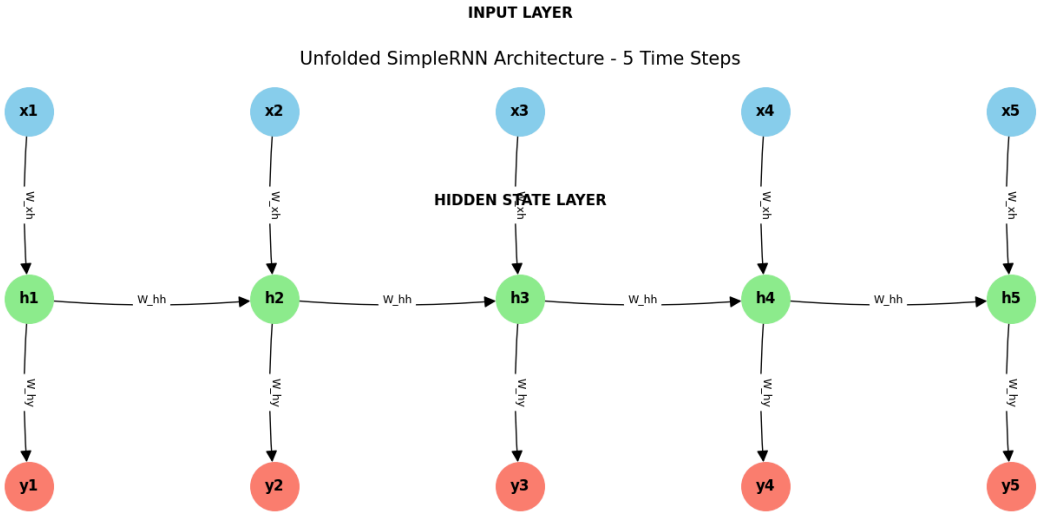
KERAS HIDDEN STATES:
[[0.1974 0.1586 0.2355]
 [0.443  0.363  0.5775]
 [0.663  0.5683 0.8192]
 [0.6832 0.6276 0.8461]
 [0.7629 0.6955 0.9021]]

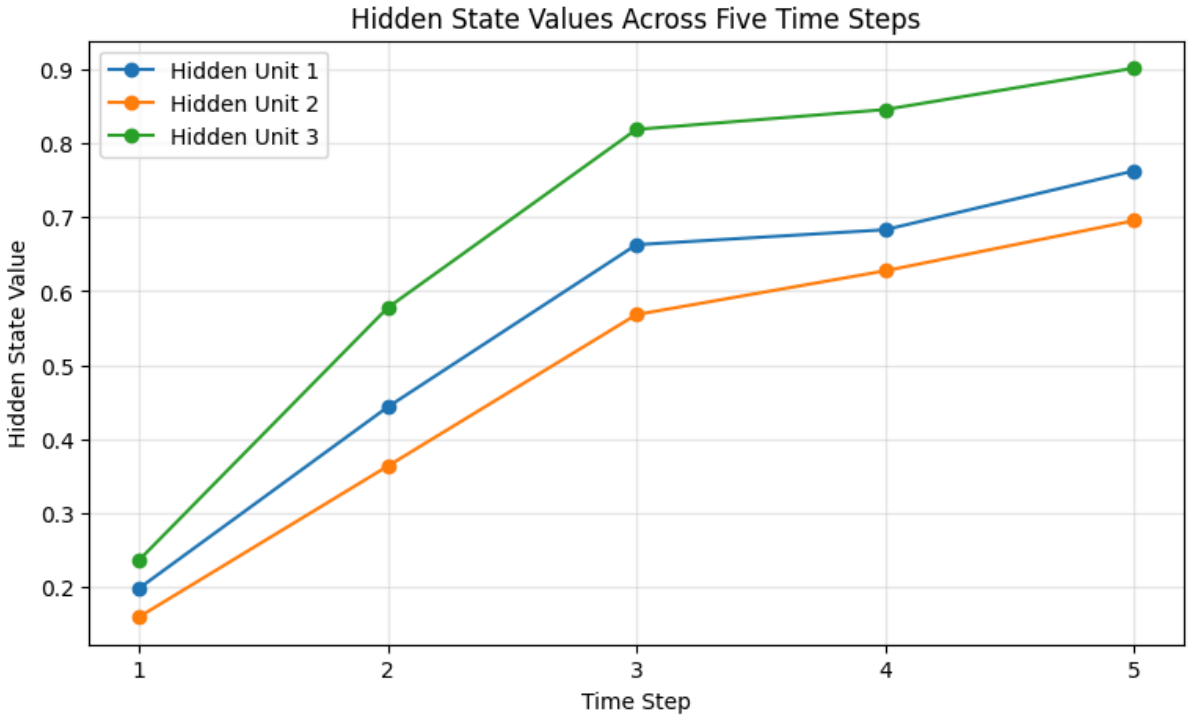
MANUAL HIDDEN STATES:
[[0.1974 0.1586 0.2355]
 [0.443  0.363  0.5775]
 [0.663  0.5683 0.8192]
 [0.6832 0.6276 0.8461]
 [0.7629 0.6955 0.9021]]

KERAS OUTPUTS:
[0.3289 0.6319 0.8951 0.9409 1.0236]

MANUAL OUTPUTS:
[0.3289 0.6319 0.8951 0.9409 1.0236]

Maximum Hidden-State Difference: 8.437310017317401e-08
Maximum Output Difference: 2.9510591526005214e-08
```





HIDDEN STATE TABLE:

	Time Step	Hidden Unit 1	Hidden Unit 2	Hidden Unit 3
0	1	0.1974	0.1586	0.2355
1	2	0.4430	0.3630	0.5775
2	3	0.6630	0.5683	0.8192
3	4	0.6832	0.6276	0.8461
4	5	0.7629	0.6955	0.9021

5. RESULT

A Simple Recurrent Neural Network with five time steps and three hidden units was implemented using TensorFlow and Keras. The recurrent architecture was unfolded using NetworkX to visualize input, hidden-state, and output connections. Hidden-state values were calculated manually using the RNN equations and verified against Keras outputs. Finally, hidden-state variations across five time steps were plotted using Matplotlib. The experiment demonstrates how an RNN processes sequential inputs and transfers information from one time step to the next using shared recurrent weights.